

SOB4ES - Extractor automatico de resultados entre ramas

Este notebook automatiza lo que se ha estado haciendo a mano hasta ahora: leer los notebooks de una rama de git (sin necesidad de hacer `checkout`, usando `git show rama:archivo`), extraer las metricas de evaluacion, el numero de variables activas, y detectar automaticamente dos problemas que ya han aparecido varias veces de forma manual:

1. **Variables que no coinciden** entre lo que dice `FEATURES_AUTORIZADAS` y lo que realmente se cargo en `X_train` (columna comentada pero no re-ejecutado).
2. **Celdas ejecutadas fuera de orden** (`execution_count` no creciente de arriba a abajo), que es la causa raiz de los resultados "pegados" de una ejecucion anterior (el bug que se detecto en `regressorchain.ipynb` y `mlp_custom_loss.ipynb`).

Compara dos ramas cualesquiera (p. ej. `main` contra `prueba1-iteracion-2`) para los 8 notebooks de modelos, y genera las correspondientes tablas comparativas.

Requisito: tener el repositorio clonado localmente (funciona sobre tu copia local de git, sin necesidad de subir nada a ningun sitio). No hace falta hacer `git checkout` de las ramas - se leen los archivos directamente del historial de git.

0.- Configuración

```
In [18]: import os
import json
import re
import pandas as pd
from io import StringIO
from IPython.display import display
import subprocess

GIT_REPO_PATH = "." # Usar '.' si ejecutas el notebook dentro del repositorio

# Modifica para definir las ramas a comparar
BRANCHES = [
    "model-prep-var",
    "model-prep-var-1",
    "model-prep-var-2",
    "model-prep-var-1-2",
    "model-prep-var-3",
    "model-prep-var-1-3",
    "model-prep-var-2-3",
    "model-prep-var-1-2-3",
    # "model-prep-var-x-y", Añade más ramas según necesites
]

BRANCHES_BARRIDO = [
    "model-prep-rs-group", # Ajusta con los nombres de las ramas
    "model-prep-rs-group-1",
]

# Notebooks a analizar
NOTEBOOKS = [
    "reg_model.ipynb",
    "rf_model.ipynb",
    "rf_multisalida.ipynb",
    "regressorchain.ipynb",
    "xgboost_model.ipynb",
    "xgb_multisalida.ipynb",
    "mlp_multisalida.ipynb",
    "mlp_custom_loss.ipynb",
]

NOTEBOOKS_SUBDIR = ""

# Lista completa de targets a extraer y comparar
TARGETS = [
    # Shannon diversity index
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z',
]

# Define los targets prioritarios que quieres destacar en la tabla comparativa
TARGETS_PRIORITARIOS = ["earthworm_shannon_z", "earthworm_richness_z"]
```

1.- Funciones de lectura desde git

`obtener_notebook_desde_git` usa `git show <rama>:<archivo>` para leer el contenido de un archivo tal y como esta en una rama concreta, sin tocar el working directory ni hacer checkout. Si el repo es remoto y no lo tienes clonado, hay una alternativa comentada al final de la celda usando la API de GitHub (`raw.githubusercontent.com`).

```
In [19]: def obtener_notebook_desde_git(repo_path, branch, filename, subdir=""):
    """Lee un .ipynb tal y como está en una rama concreta, via `git show`, sin checkout."""
    ruta_relativa = os.path.join(subdir, filename) if subdir else filename
```

```

try:
    resultado = subprocess.run(
        ["git", "-C", repo_path, "show", f"{branch}:{ruta_relativa}"],
        capture_output=True,
        text=True,
        check=True,
    )
except subprocess.CalledProcessError as e:
    print(
        f" [ERROR] No se pudo leer {filename} en la rama {branch}: {e.stderr.strip()}"
    )
    return None
return json.loads(resultado.stdout)

def listar_ramas(repo_path):
    """Útil para comprobar el nombre exacto de las ramas disponibles (locales y remotas)."""
    resultado = subprocess.run(
        ["git", "-C", repo_path, "branch", "-a"],
        capture_output=True,
        text=True,
    )
    print(resultado.stdout)

```

2.- Funciones de extracción

- `extraer_features_activas`: parsea la lista `FEATURES_AUTORIZADAS` y separa las líneas comentadas (excluidas) de las activas.
- `extraer_shape_xtrain`: busca el print de `X_train`: (filas, columnas) para saber cuantas variables se usaron realmente en el entrenamiento.
- `detectar_celdas_desordenadas`: compara el `execution_count` de las celdas de código en el orden en que aparecen en el notebook; si no es creciente, señala un posible problema de ejecución (resultados de una corrida anterior, no de la actual).
- `extraer_metricas_targets`: busca en los outputs de texto, **a partir del marcador "Evaluacion final sobre eval.csv"**, las líneas con `R2/RMSE/MAE` para los targets pedidos. Restringir la búsqueda a ese bloque es importante: sin eso, el regex puede confundirse con el `R2` de entrenamiento/CV impreso en las celdas de tuning (mucho más alto por sobreajuste) y dar una lectura falsa. Soporta dos formatos: modelos single-target (`R2+RMSE+MAE` por target) y modelos multiclase (solo `R2` por target, con `RMSE/MAE` únicamente a nivel global).

```

In [20]: def extraer_features_activas(nb_json):
    for c in nb_json["cells"]:
        src = "".join(c.get("source", []))
        if "FEATURES_AUTORIZADAS =" in src:
            m = re.search(r"FEATURES_AUTORIZADAS\s*=\s*[^(.?)\]", src, re.S)
            if not m:
                continue
            lineas = [l.strip() for l in m.group(1).split("\n") if l.strip()]
            activas, excluidas = [], []
            for linea in lineas:
                nombre_m = re.search(r"'([^\s]+)'", linea)
                if not nombre_m:
                    continue
                nombre = nombre_m.group(1)
                if linea.startswith("#"):
                    excluidas.append(nombre)
                else:
                    activas.append(nombre)
            return activas, excluidas

def extraer_shape_xtrain(nb_json):
    for c in nb_json["cells"]:
        for out in c.get("outputs", []):
            texto = "".join(out.get("text", []))
            m = re.search(r"X_train:\s*((\d+)\s*(\d+)\s*)", texto)
            if m:
                return int(m.group(1)), int(m.group(2))
    return None, None

def detectar_celdas_desordenadas(nb_json):
    """Devuelve una lista de avisos si el execution_count no es creciente."""
    avisos = []
    ultimo_exec = None
    for i, c in enumerate(nb_json["cells"]):
        if c.get("cell_type") != "code":
            continue
        exec_count = c.get("execution_count")
        if exec_count is None:
            continue
        if ultimo_exec is not None and exec_count < ultimo_exec:
            avisos.append(
                f"celda #{i} tiene execution_count={exec_count}, "
                f"menor que una celda anterior ({ultimo_exec}) → posible resultado obsoleto"
            )
        ultimo_exec = exec_count
    return avisos

MARCADOR_EVAL = "Evaluacion final sobre eval.csv"

def extraer_metricas_targets(nb_json, targets):
    """Extrae R2/RMSE/MAE de la evaluación final para cada target."""
    resultados = {}
    patron_completo = {
        t: re.compile(
            rf"{re.escape(t)}\s+([\d-0-9.]+\s+([\d-0-9.]+\s+([\d-0-9.]+\s+([\d-0-9.]+\s+
        )
        for t in targets
    }
    patron_solo_r2 = {
        t: re.compile(rf"{re.escape(t)}\s+([\d-0-9.]+\s*$", re.M)
        for t in targets
    }
    patron_global = re.compile(
        r"R2\s+global:\s*([\d-0-9.]+).*?RMSE\s+global:\s*([\d-0-9.]+).*?MAE\s+global:\s*([\d-0-9.]+)",
        re.S,
    )

```

```

)

for c in nb_json["cells"]:
    # OJO: algunos notebooks (rf_multisalida, xgb_multisalida, regressorchain)
    # imprimen la cabecera "Evaluacion final..." y el bloque de métricas en
    # dos print() distintos, que Jupyter guarda como dos outputs SEPARADOS
    # dentro de la misma celda. Por eso hay que concatenar todos los outputs
    # de la celda ANTES de buscar el marcador, en vez de mirarlos uno a uno.
    texto_completo = "".join(
        "".join(out.get("text", []))
        for out in c.get("outputs", [])
        if out.get("output_type") == "stream"
    )
    if MARCADOR_EVAL not in texto_completo:
        continue
    texto = texto_completo[texto_completo.index(MARCADOR_EVAL):]

    g = patron_global.search(texto)
    global_metrics = (
        {
            "r2_global": float(g.group(1)),
            "rmse_global": float(g.group(2)),
            "mae_global": float(g.group(3)),
        }
        if g
        else None
    )

    for t in targets:
        if t in resultados:
            continue
        m = patron_completo[t].search(texto)
        if m:
            resultados[t] = {
                "r2": float(m.group(1)),
                "rmse": float(m.group(2)),
                "mae": float(m.group(3)),
            }
            continue
        m2 = patron_solo_r2[t].search(texto)
        if m2:
            resultados[t] = {
                "r2": float(m2.group(1)),
                "rmse": None,
                "mae": None,
                "global": global_metrics,
            }
    return resultados

def extraer_random_state(nb_json):
    """Busca el random_state usado para entrenar. Prioriza una constante de
    configuración tipo 'RANDOM_STATE = N'; si no existe, coge la primera
    aparición de 'random_state=N' que encuentre en el código."""
    codigo_completo = "\n".join(
        "".join(c.get("source", []))
        for c in nb_json["cells"]
        if c.get("cell_type") == "code"
    )
    m = re.search(r"RANDOM_STATE\s*=\s*(\d+)", codigo_completo)
    if m:
        return int(m.group(1))
    m2 = re.search(r"random_state\s*=\s*(\d+)", codigo_completo)
    if m2:
        return int(m2.group(1))
    return None

```

```

In [21]: def extraer_ruta_csv_barrido(nb_json):
    """Deduce la ruta del CSV de resultados del barrido de random_state
    leyendo el propio código del notebook (MODELS_DIR + dir_barrido +
    el nombre de archivo del .to_csv), en vez de asumir un nombre fijo tipo
    'ridge_barrido_rs.csv' -- así sirve igual para RF, XGBoost, etc."""
    codigo = "\n".join(
        "".join(c.get("source", []))
        for c in nb_json["cells"]
        if c.get("cell_type") == "code"
    )

    m_dir = re.search(r"MODELS_DIR\s*=\s*['\"]([^\"]+)['\"]", codigo)
    models_dir = m_dir.group(1) if m_dir else ""

    m_sub = re.search(
        r"dir_barrido\s*=\s*os.path.join\(\\s*MODELS_DIR\s*,\s*['\"]([^\"]+)['\"]\s*\)", codigo
    )
    if not m_sub:
        return None
    subdir = m_sub.group(1)

    m_file = re.search(
        r"\\.to_csv\(\\s*os.path.join\(\\s*dir_barrido\s*,\s*['\"]([^\"]+)['\"]\s*\)", codigo
    )
    if not m_file:
        return None
    filename = m_file.group(1)

    return "/".join(p.strip("/") for p in [models_dir, subdir, filename] if p)

def obtener_csv_desde_git(repo_path, branch, ruta_csv):
    """Lee un CSV vía 'git show', igual que obtener_notebook_desde_git pero
    parseando CSV en vez de JSON."""
    try:
        resultado = subprocess.run(
            ["git", "-C", repo_path, "show", f"{branch}:{ruta_csv}"],
            capture_output=True, text=True, check=True,
        )
    except subprocess.CalledProcessError as e:
        print(f" [ERROR] No se pudo leer {ruta_csv} en la rama {branch}: {e.stderr.strip()}")
        return None

```

```

        return pd.read_csv(StringIO(resultado.stdout))

def extraer_rango_esperado_rs(nb_json):
    """Lee 'RANDOM_STATES = range(a, b)' o 'range(a, b, paso)' del código y
    devuelve cuántas semillas se esperan, para poder comparar contra las
    filas que realmente tiene el CSV del barrido."""
    codigo = "\n".join(
        "".join(c.get("source", []))
        for c in nb_json["cells"]
        if c.get("cell_type") == "code"
    )
    m = re.search(
        r"RANDOM_STATES\s*=\s*range\(\\s*(\\d+)\s*,\\s*(\\d+)\s*(?:,\\s*(\\d+)\s*)?\)", codigo
    )
    if not m:
        return None
    inicio, fin = int(m.group(1)), int(m.group(2))
    paso = int(m.group(3)) if m.group(3) else 1
    return len(range(inicio, fin, paso))

```

3.- Extracción de notebooks y por rama

```

In [22]: def analizar_notebook(repo_path, branch, filename, subdir, targets):
nb_json = obtener_notebook_desde_git(repo_path, branch, filename, subdir)
if nb_json is None:
    return None

activas, excluidas = extraer_features_activas(nb_json)
filas, columnas = extraer_shape_xtrain(nb_json)
metricas = extraer_metricas_targets(nb_json, targets)
avisos_orden = detectar_celdas_desordenadas(nb_json)
random_state = extraer_random_state(nb_json)

return {
    "n_features_lista": len(activas),
    "features_excluidas": excluidas,
    "x_train_shape": (filas, columnas),
    "coherente": (
        (columnas == len(activas)) if columnas is not None else None
    ),
    "metricas": metricas,
    "avisos_orden": avisos_orden,
    "random_state": random_state,
}

resultados_por_rama = {}

for branch in BRANCHES:
    print(f"Leyendo notebooks de la rama: {branch}...")
    resultados_por_rama[branch] = {}
    for nb_name in NOTEBOOKS:
        print(f"  {nb_name}")
        resultados_por_rama[branch][nb_name] = analizar_notebook(
            GIT_REPO_PATH,
            branch,
            nb_name,
            NOTEBOOKS_SUBDIR,
            TARGETS,
        )
    print()

```

Leyendo notebooks de la rama: model-prep-var...

```
reg_model.ipynb
rf_model.ipynb
rf_multisalida.ipynb
regressorchain.ipynb
xgboost_model.ipynb
xgb_multisalida.ipynb
mlp_multisalida.ipynb
mlp_custom_loss.ipynb
```

Leyendo notebooks de la rama: model-prep-var-1...

```
reg_model.ipynb
rf_model.ipynb
rf_multisalida.ipynb
regressorchain.ipynb
xgboost_model.ipynb
xgb_multisalida.ipynb
mlp_multisalida.ipynb
mlp_custom_loss.ipynb
```

Leyendo notebooks de la rama: model-prep-var-2...

```
reg_model.ipynb
rf_model.ipynb
rf_multisalida.ipynb
regressorchain.ipynb
xgboost_model.ipynb
xgb_multisalida.ipynb
mlp_multisalida.ipynb
mlp_custom_loss.ipynb
```

Leyendo notebooks de la rama: model-prep-var-1-2...

```
reg_model.ipynb
rf_model.ipynb
rf_multisalida.ipynb
regressorchain.ipynb
xgboost_model.ipynb
xgb_multisalida.ipynb
mlp_multisalida.ipynb
mlp_custom_loss.ipynb
```

Leyendo notebooks de la rama: model-prep-var-3...

```
reg_model.ipynb
rf_model.ipynb
rf_multisalida.ipynb
regressorchain.ipynb
xgboost_model.ipynb
xgb_multisalida.ipynb
mlp_multisalida.ipynb
mlp_custom_loss.ipynb
```

Leyendo notebooks de la rama: model-prep-var-1-3...

```
reg_model.ipynb
rf_model.ipynb
rf_multisalida.ipynb
regressorchain.ipynb
xgboost_model.ipynb
xgb_multisalida.ipynb
mlp_multisalida.ipynb
mlp_custom_loss.ipynb
```

Leyendo notebooks de la rama: model-prep-var-2-3...

```
reg_model.ipynb
rf_model.ipynb
rf_multisalida.ipynb
regressorchain.ipynb
xgboost_model.ipynb
xgb_multisalida.ipynb
mlp_multisalida.ipynb
mlp_custom_loss.ipynb
```

Leyendo notebooks de la rama: model-prep-var-1-2-3...

```
reg_model.ipynb
rf_model.ipynb
rf_multisalida.ipynb
regressorchain.ipynb
xgboost_model.ipynb
xgb_multisalida.ipynb
mlp_multisalida.ipynb
mlp_custom_loss.ipynb
```

4. Validaciones automaticas

Antes de comparar metricas, se comprueba automaticamente lo que hasta ahora se ha ido revisando a mano:

- ¿El numero de columnas de `X_train` coincide con el numero de variables activas en `FEATURES_AUTORIZADAS` ?
- ¿Hay celdas con `execution_count` fuera de orden (posible resultado obsoleto)?
- ¿Los valores de los targets prioritarios son sospechosamente identicos entre ambas ramas (posible notebook no re-ejecutado)?

```
In [23]: print("=" * 100)
print("VALIDACIONES AUTOMÁTICAS")
print("=" * 100)

base_branch = BRANCHES[0]

for nb_name in NOTEBOOKS:
    print(f"\n[{nb_name}]")

    rs_por_notebook = {}
    for branch in BRANCHES:
        r = resultados_por_rama.get(branch, {}).get(nb_name)
        if r is None:
            print(f" [ERROR - {branch}] No se pudo leer el notebook.")
            continue

    filas, columnas = r["x_train_shape"]
    rs_por_notebook[branch] = r.get("random_state")
```

```

# con estas pruebas NO se eliminan variables: cualquier diferencia
# en el número de columnas es sospechosa
if r["coherente"] is False:
    print(
        f" [AVISO - {branch}] Se esperaban las mismas variables que en el resto "
        f"de ramas (esta prueba no elimina variables), pero X_train tiene {columnas}: "
        f"columnas y FEATURES_AUTORIZADAS tiene {r['n_features_lista']} activas → revisar"
    )
if r["avisos_orden"]:
    print(f" [AVISO - {branch}] Celdas ejecutadas fuera de orden:")
    for a in r["avisos_orden"]:
        print(f" - {a}")

m = re.search(r"(\d+)\s*$", branch)
if m and r.get("random_state") is not None:
    rs_esperado = int(m.group(1))
    if r["random_state"] != rs_esperado:
        print(
            f" [AVISO - {branch}] El nombre de la rama sugiere random_state={rs_esperado}, "
            f"pero el notebook usa random_state={r['random_state']}"
        )

valores_rs = set(v for v in rs_por_notebook.values() if v is not None)
if len(valores_rs) > 1:
    print(f" [AVISO] Los notebooks de esta rama no usan todos el mismo random_state: {rs_por_notebook}")

r_base = resultados_por_rama.get(base_branch, {}).get(nb_name)
if r_base:
    for comp_branch in BRANCHES[1:]:
        r_comp = resultados_por_rama.get(comp_branch, {}).get(nb_name)
        if not r_comp:
            continue
        for t in TARGETS_PRIORITARIOS:
            m_base = r_base["metricas"].get(t)
            m_comp = r_comp["metricas"].get(t)
            if (
                m_base
                and m_comp
                and m_base.get("r2") == m_comp.get("r2")
            ):
                print(
                    f" [AVISO] {t}: R2 IDENTICO en '{base_branch}' y '{comp_branch}' ({m_base['r2']}) "
                    f"→ revisar si se re-ejecutó de verdad"
                )

```

```
=====
VALIDACIONES AUTOMÁTICAS
=====

[reg_model.ipynb]
[AVISO - model-prep-var-1] El nombre de la rama sugiere random_state=1, pero el notebook usa random_state=42
[AVISO - model-prep-var-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42

[rf_model.ipynb]
[AVISO - model-prep-var-1] El nombre de la rama sugiere random_state=1, pero el notebook usa random_state=42
[AVISO - model-prep-var-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42

[rf_multisalida.ipynb]
[AVISO - model-prep-var-1] El nombre de la rama sugiere random_state=1, pero el notebook usa random_state=42
[AVISO - model-prep-var-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42

[regressorchain.ipynb]
[AVISO - model-prep-var-1] El nombre de la rama sugiere random_state=1, pero el notebook usa random_state=42
[AVISO - model-prep-var-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42

[xgboost_model.ipynb]
[AVISO - model-prep-var-1] El nombre de la rama sugiere random_state=1, pero el notebook usa random_state=42
[AVISO - model-prep-var-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42

[xgb_multisalida.ipynb]
[AVISO - model-prep-var-1] El nombre de la rama sugiere random_state=1, pero el notebook usa random_state=42
[AVISO - model-prep-var-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42

[mlp_multisalida.ipynb]
[AVISO - model-prep-var-1] El nombre de la rama sugiere random_state=1, pero el notebook usa random_state=42
[AVISO - model-prep-var-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42

[mlp_custom_loss.ipynb]
[AVISO - model-prep-var-1] El nombre de la rama sugiere random_state=1, pero el notebook usa random_state=42
[AVISO - model-prep-var-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2] El nombre de la rama sugiere random_state=2, pero el notebook usa random_state=42
[AVISO - model-prep-var-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
[AVISO - model-prep-var-1-2-3] El nombre de la rama sugiere random_state=3, pero el notebook usa random_state=42
```

4.1.- Etiquetado automático de ramas y utilidades HTML (para la sección 5)

Las ramas de `BRANCHES` pueden pertenecer a dos tipos de prueba distintos:

- **Eliminación de variables:** mismo `random_state`, pero cambia el número de predictoras (columnas de `X_train`).
- **Sensibilidad al `random_state`:** mismo número de predictoras, pero cambia el `random_state`.

Las ramas de `BRANCHES_BARRIDO` (sección 8) se excluyen de este cálculo, porque esas ya se leen y etiquetan aparte a partir de los CSV del barrido. Para el resto, se detecta automáticamente cuál de los dos criterios (o ambos) varía realmente entre ramas, y se usa para etiquetarlas.

Esto evita el problema de antes: si todas las ramas comparadas comparten el mismo `random_state` (por tratarse de una prueba de eliminación de variables), etiquetarlas solo por `rs=...` genera la misma etiqueta para todas, y al construir la tabla las columnas se pisan entre sí -- perdiendo silenciosamente los datos del resto de las ramas. Como red de seguridad adicional, si dos ramas acabasen teniendo la misma etiqueta (caso raro en que ni el `rs` ni el nº de variables las distinguen), se añade el nombre de la rama entre corchetes para que nunca haya colisiones.

```
In [24]: from collections import Counter
        from IPython.display import display, HTML

        def _valor_mas_frecuente(contador):
            return contador.most_common(1)[0][0] if contador else None

        def calcular_criterio_ramas(branches):
            """Para las ramas que NO forman parte del barrido de random_state
            (BRANCHES_BARRIDO), determina automáticamente si difieren entre si por:
            - el random_state usado (prueba de sensibilidad al random_state), o
            - el número de predictoras / variables activas (prueba de
              eliminación de variables),
            o por ambas cosas a la vez. Devuelve el criterio detectado junto con el
            valor "representativo" (el más frecuente entre los notebooks) de cada
```

```

rama, para poder etiquetarlas sin ambigüedad.
"""
ramas_a_mirar = [b for b in branches if b not in BRANCHES_BARRIDO]

rs_por_rama, nvars_por_rama = {}, {}
for branch in ramas_a_mirar:
    conteo_rs, conteo_nvars = Counter(), Counter()
    for nb_name in NOTEBOOKS:
        res = resultados_por_rama.get(branch, {}).get(nb_name)
        if not res:
            continue
        if res.get("random_state") is not None:
            conteo_rs[res["random_state"]] += 1

        # n° real de columnas de X_train si lo tenemos; si no, n° de
        # variables activas en FEATURES_AUTORIZADAS
        n_vars = None
        if res.get("x_train_shape"):
            n_vars = res["x_train_shape"][1]
        if n_vars is None:
            n_vars = res.get("n_features_lista")
        if n_vars is not None:
            conteo_nvars[n_vars] += 1

    rs_por_rama[branch] = _valor_mas_frecuente(conteo_rs)
    nvars_por_rama[branch] = _valor_mas_frecuente(conteo_nvars)

rs_distintos = set(v for v in rs_por_rama.values() if v is not None)
nvars_distintos = set(v for v in nvars_por_rama.values() if v is not None)

rs_varia = len(rs_distintos) > 1
nvars_varia = len(nvars_distintos) > 1

if rs_varia and nvars_varia:
    criterio = "ambos"
elif rs_varia:
    criterio = "rs"
elif nvars_varia:
    criterio = "n_vars"
else:
    criterio = "ninguno" # no se detecta variación → se usará el nombre de rama

return criterio, rs_por_rama, nvars_por_rama

def obtener_label_rama(branch, criterio, rs_por_rama, nvars_por_rama):
    """Etiqueta una rama según el criterio que realmente la diferencia del
    resto (random_state y/o n° de predictoras). Las ramas del barrido
    (BRANCHES_BARRIDO) se dejan con su nombre tal cual, porque se etiquetan
    aparte en la sección 8."""
    if branch in BRANCHES_BARRIDO:
        return branch

    partes = []
    if criterio in ("rs", "ambos") and rs_por_rama.get(branch) is not None:
        partes.append(f"rs={rs_por_rama[branch]}")
    if criterio in ("n_vars", "ambos") and nvars_por_rama.get(branch) is not None:
        partes.append(f"n_vars={nvars_por_rama[branch]}")

    if not partes:
        return branch

    return ", ".join(partes)

CRITERIO_RAMAS, RS_POR_RAMAS, NVARSPOR_RAMAS = calcular_criterio_ramas(BRANCHES)

LABELS_RAMAS = {
    branch: obtener_label_rama(branch, CRITERIO_RAMAS, RS_POR_RAMAS, NVARSPOR_RAMAS)
    for branch in BRANCHES
}

# Red de seguridad: si dos ramas comparten la misma etiqueta, se desambigua
# añadiendo el nombre de rama, para no perder nunca una columna.
_conteo_labels = Counter(LABELS_RAMAS.values())
LABELS_RAMAS = {
    branch: (label if _conteo_labels[label] == 1 else f"{label} [{branch}]")
    for branch, label in LABELS_RAMAS.items()
}

print(f"Criterio detectado para diferenciar las ramas de BRANCHES: '{CRITERIO_RAMAS}'")
for branch in BRANCHES:
    print(f" {branch} → {LABELS_RAMAS[branch]}")

# --- Utilidades para tablas HTML (sección 5) -----

ESTILO_HTML_TABLAS = (
    "<style>"
    "table{border-collapse:collapse;margin-bottom:30px;font-family:sans-serif;font-size:13px;} "
    "th,td{border:1px solid #ccc;padding:4px 8px;text-align:right;} "
    "th{background:#f0f0f0;} "
    "td:first-child,th:first-child{text-align:left;} "
    "h1,h2,h3{font-family:sans-serif;}"
    "</style>"
)

def render_tabla_html(df, na_rep="-", float_format="{:.4f}"):
    """Convierte un DataFrame a HTML con formato consistente para toda la
    sección 5 (mismo separador de miles/decimales y el mismo valor para
    los NaN en todas las tablas)."""
    return df.to_html(
        index=False,
        na_rep=na_rep,
        float_format=lambda x: float_format.format(x),
    )

```



```
def mostrar_tabla_html(df, titulo):
    """Muestra un DataFrame como tabla HTML con estilo, directamente en el
    notebook."""
    display(HTML(f"<h3>{titulo}</h3>{ESTILO_HTML_TABLAS}{render_tabla_html(df)}"))

def guardar_tabla_html(html_body, ruta, titulo=""):
    """Envuelve uno o varios fragmentos HTML de tabla en un documento HTML
    completo (con estilo) y lo guarda en disco."""
    html_completo = (
        "<html><head><meta charset='utf-8'>" + ESTILO_HTML_TABLAS + "</head><body>"
        + (f"<h1>{titulo}</h1>" if titulo else "")
        + html_body
        + "</body></html>"
    )
    with open(ruta, "w", encoding="utf-8") as f:
        f.write(html_completo)
    return ruta
```

Criterio detectado para diferenciar las ramas de BRANCHES: 'n_vars'

model-prep-var	→	n_vars=34
model-prep-var-1	→	n_vars=33 [model-prep-var-1]
model-prep-var-2	→	n_vars=33 [model-prep-var-2]
model-prep-var-1-2	→	n_vars=32 [model-prep-var-1-2]
model-prep-var-3	→	n_vars=33 [model-prep-var-3]
model-prep-var-1-3	→	n_vars=32 [model-prep-var-1-3]
model-prep-var-2-3	→	n_vars=32 [model-prep-var-2-3]
model-prep-var-1-2-3	→	n_vars=31

5- Tablas comparativas

En esta sección se generarán múltiples tablas para facilitar la comparación de resultados.

```
In [25]:
filas_tabla = []
base_branch = BRANCHES[0]

for nb_name in NOTEBOOKS:
    fila = {"Notebook": nb_name}

    for t in TARGETS_PRIORITARIOS:
        # Obtener R2 de la rama base
        r_base = resultados_por_rama.get(base_branch, {}).get(nb_name)
        m_base = r_base["metricas"].get(t) if r_base else None
        r2_base = m_base["r2"] if m_base else None

        fila[f"{t}_R2 ({LABELS_RAMAS[base_branch])}] = r2_base

        # R2 y Delta para las ramas a comparar
        for branch in BRANCHES[1:]:
            r_branch = resultados_por_rama.get(branch, {}).get(nb_name)
            m_branch = r_branch["metricas"].get(t) if r_branch else None
            r2_val = m_branch["r2"] if m_branch else None

            fila[f"{t}_R2 ({LABELS_RAMAS[branch])}] = r2_val

            delta = (
                round(r2_val - r2_base, 4)
                if (r2_val is not None and r2_base is not None)
                else None
            )
            fila[f"{t}_delta ({LABELS_RAMAS[branch])}] = delta

        filas_tabla.append(fila)

df_comparativa = pd.DataFrame(filas_tabla)

print("=" * 100)
print(f"5. TABLA COMPARATIVA POR TARGETS PRIORITARIOS (Base: {LABELS_RAMAS[base_branch])}")
print("=" * 100)

html_comparativa = render_tabla_html(df_comparativa)
mostrar_tabla_html(
    df_comparativa,
    f"Comparación entre {len(BRANCHES)} ramas (Base: {LABELS_RAMAS[base_branch])}",
)

# Guardar resultado en CSV y en HTML
os.makedirs("output/comparativas", exist_ok=True)
nombre_salida = f"comparativa_{'_vs_'}.join(BRANCHES)}.replace('/', '-'))

ruta_csv = f"output/comparativas/{nombre_salida}.csv"
df_comparativa.to_csv(ruta_csv, index=False)
print(f"\nGuardado en: {ruta_csv}")

ruta_html = f"output/comparativas/{nombre_salida}.html"
guardar_tabla_html(
    html_comparativa,
    ruta_html,
    titulo=f"Comparación entre ramas (Base: {LABELS_RAMAS[base_branch])}",
)
print(f"Guardado en: {ruta_html}")
```

=====

5. TABLA COMPARATIVA POR TARGETS PRIORITARIOS (Base: n_vars=34)

=====

Comparación entre 8 ramas (Base: n_vars=34)

Notebook	earthworm_shannon_z_R2 (n_vars=34)	earthworm_shannon_z_R2 (n_vars=33 [model-prep- var-1])	earthworm_shannon_z_delta (n_vars=33 [model-prep-var- 1])	earthworm_shannon_z_R2 (n_vars=33 [model-prep- var-2])	earthworm_shannon_z_delta (n_vars=33 [model-prep-var- 2])	earthworm_sf (n_vars=32)
reg_model.ipynb	0.2395	0.2434	0.0039	0.2401	0.0006	
rf_model.ipynb	0.5041	0.5014	-0.0027	0.4995	-0.0046	
rf_multisalida.ipynb	0.4160	0.4045	-0.0115	0.4113	-0.0047	
regressorchain.ipynb	0.4876	0.4890	0.0014	0.4812	-0.0064	
xgboost_model.ipynb	0.4946	0.4931	-0.0015	0.4906	-0.0040	
xgb_multisalida.ipynb	0.5375	0.5472	0.0097	0.5423	0.0048	
mlp_multisalida.ipynb	0.3583	0.3504	-0.0079	0.3463	-0.0120	
mlp_custom_loss.ipynb	0.3438	0.3569	0.0131	0.4070	0.0632	

Guardado en: output/comparativas/comparativa_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_mo
del-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.csv
Guardado en: output/comparativas/comparativa_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_mo
del-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

```
In [26]: import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")

def construir_df_largo_barrido():
    filas = []
    for branch in BRANCHES:
        for nb_name in NOTEBOOKS:
            res = resultados_por_rama.get(branch, {}).get(nb_name)
            if not res:
                continue
            rs = res.get("random_state")
            m_dict = res.get("metricas", {})

            # R2 global (mismo criterio que en la tabla general de la sección 5.1)
            r2_glob = None
            for t_info in m_dict.values():
                if t_info and t_info.get("global"):
                    r2_glob = t_info["global"].get("r2_global")
                    break
            if r2_glob is None and m_dict:
                r2_vals = [v["r2"] for v in m_dict.values() if v and v.get("r2") is not None]
                if r2_vals:
                    r2_glob = sum(r2_vals) / len(r2_vals)
            if r2_glob is not None:
                filas.append({"Notebook": nb_name, "branch": branch, "random_state": rs, "target": "GLOBAL", "r2": r2_glob})

            for t in TARGETS_PRIORITARIOS:
                info = m_dict.get(t)
                if info and info.get("r2") is not None:
                    filas.append({"Notebook": nb_name, "branch": branch, "random_state": rs, "target": t, "r2": info["r2"]})

    return pd.DataFrame(filas)

df_barrido = construir_df_largo_barrido()
print(f"Filas recogidas: {len(df_barrido)} (esperado: hasta {len(BRANCHES)} semillas x {len(NOTEBOOKS)} notebooks x 3 targets)")
df_barrido.head()
```

Filas recogidas: 192 (esperado: hasta 8 semillas x 8 notebooks x 3 targets)

```
Out[26]:
```

	Notebook	branch	random_state	target	r2
0	reg_model.ipynb	model-prep-var	42	GLOBAL	-0.016376
1	reg_model.ipynb	model-prep-var	42	earthworm_shannon_z	0.239500
2	reg_model.ipynb	model-prep-var	42	earthworm_richness_z	0.278900
3	rf_model.ipynb	model-prep-var	42	GLOBAL	0.090362
4	rf_model.ipynb	model-prep-var	42	earthworm_shannon_z	0.504100

5.1.- Tabla de compración general

```
In [27]: filas_general = []
base_branch = BRANCHES[0]

for nb_name in NOTEBOOKS:
    fila = {"Notebook": nb_name}

    for branch in BRANCHES:
        res = resultados_por_rama.get(branch, {}).get(nb_name)
        label = LABELS_RAMAS[branch]

        if res:
            # Dimensiones de X_train
            shape_str = (
                f"{res['x_train_shape'][0]}x{res['x_train_shape'][1]}"
                if res["x_train_shape"][0]
                else "N/A"
            )
            fila[f"N_Vars ({label})"] = res["n_features_lista"]
            fila[f"Shape ({label})"] = shape_str

            # Extracción de métricas globales
            m_dict = res.get("metricas", {})
            r2_glob, rmse_glob, mae_glob = None, None, None

            # 1. Intentar obtener 'global' si el notebook es multisalida
            for t_info in m_dict.values():
                if t_info and t_info.get("global"):
                    r2_glob = t_info["global"].get("r2_global")
                    rmse_glob = t_info["global"].get("rmse_global")
                    mae_glob = t_info["global"].get("mae_global")
```

```

        break

# 2. Si es single-target, promediar el R2 de los targets evaluados
if r2_glob is None and m_dict:
    r2_vals = [
        v["r2"]
        for v in m_dict.values()
        if v and v.get("r2") is not None
    ]
    if r2_vals:
        r2_glob = round(sum(r2_vals) / len(r2_vals), 4)

fila[f"R2_Global ({label})"] = r2_glob
if rmse_glob is not None:
    fila[f"RMSE_Global ({label})"] = rmse_glob
if mae_glob is not None:
    fila[f"MAE_Global ({label})"] = mae_glob

# Calcular Delta R2 Global respecto al baseline
if branch != base_branch:
    r2_base = fila.get(f"R2_Global ({LABELS_RAMAs[base_branch]})")
    fila[f"Delta_R2_Global ({label})"] = (
        round(r2_glob - r2_base, 4)
        if (r2_glob is not None and r2_base is not None)
        else None
    )
else:
    fila[f"N_Vars ({label})"] = "Error"
    fila[f"R2_Global ({label})"] = None

filas_general.append(fila)

df_general = pd.DataFrame(filas_general)

print("=" * 100)
print(f"5.1. TABLA COMPARATIVA GENERAL (Base: {LABELS_RAMAs[base_branch]})")
print("=" * 100)

html_general = render_tabla_html(df_general)
mostrar_tabla_html(
    df_general, f"Tabla comparativa general (Base: {LABELS_RAMAs[base_branch]})"
)

# Guardar CSV y HTML
os.makedirs("output/comparativas", exist_ok=True)
nombre_base = f"general_{'_vs_'.join(BRANCHES)}.replace("/", "-")

ruta_csv_gen = f"output/comparativas/{nombre_base}.csv"
df_general.to_csv(ruta_csv_gen, index=False)
print(f"\nGuardado en: {ruta_csv_gen}")

ruta_html_gen = f"output/comparativas/{nombre_base}.html"
guardar_tabla_html(
    html_general,
    ruta_html_gen,
    titulo=f"Tabla comparativa general (Base: {LABELS_RAMAs[base_branch]})",
)
print(f"Guardado en: {ruta_html_gen}")

```

=====

5.1. TABLA COMPARATIVA GENERAL (Base: n_vars=34)

=====

Tabla comparativa general (Base: n_vars=34)

Notebook	N_Vars (n_vars=34)	Shape (n_vars=34)	R2_Global (n_vars=34)	N_Vars (n_vars=33 [model- prep-var- 1])	Shape (n_vars=33 [model- prep-var- 1])	R2_Global (n_vars=33 [model- prep-var- 1])	Delta_R2_Global (n_vars=33 [model-prep-var- 1])	N_Vars (n_vars=33 [model- prep-var- 2])	Shape (n_vars=33 [model- prep-var- 2])	R2_Global (n_vars=33 [model- prep-var- 2])	Delta_R2_Global (n_vars=33 [model-prep-var- 2])	?
reg_model.ipynb	34	299x34	-0.0164	33	299x33	-0.0159	0.0005	33	299x33	-0.0131	0.0033	
rf_model.ipynb	34	299x34	0.0904	33	299x33	0.0886	-0.0018	33	299x33	0.0937	0.0033	
rf_multisalida.ipynb	34	299x34	0.0964	33	299x33	0.0899	-0.0065	33	299x33	0.0913	-0.0051	
regressorchain.ipynb	34	299x34	0.0809	33	299x33	0.0781	-0.0028	33	299x33	0.0795	-0.0014	
xgboost_model.ipynb	34	299x34	0.0670	33	299x33	0.0690	0.0020	33	299x33	0.0680	0.0010	
xgb_multisalida.ipynb	34	299x34	0.0835	33	299x33	0.0779	-0.0056	33	299x33	0.0833	-0.0002	
mlp_multisalida.ipynb	34	299x34	0.0017	33	299x33	-0.0093	-0.0110	33	299x33	-0.0795	-0.0812	
mlp_custom_loss.ipynb	34	299x34	-0.0978	33	299x33	-0.0212	0.0766	33	299x33	-0.0804	0.0174	

Guardado en: output/comparativas/general_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.csv

Guardado en: output/comparativas/general_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

5.2.- Comparación por targets

```

In [28]: base_branch = BRANCHES[0]
tablas_por_target = {}
html_partes = []

os.makedirs("output/comparativas", exist_ok=True)

for target in TARGETS:
    filas_target = []

    for nb_name in NOTEBOOKS:
        fila = {"Notebook": nb_name}

        for branch in BRANCHES:
            res = resultados_por_rama.get(branch, {}).get(nb_name)
            m = (
                res["metricas"].get(target)
                if (res and res.get("metricas"))
            )

```

```

        else None
    )
    # Etiqueta generalizada (rs y/o n° de predictoras según lo que
    # realmente varíe entre ramas) -- así ninguna rama se pisa con
    # otra al construir las columnas de la tabla.
    fila[f"R2 ({LABELS_RAMAS[branch]})"] = m["r2"] if m else None

    filas_target.append(fila)

df_target = pd.DataFrame(filas_target)

cols_r2 = [c for c in df_target.columns if c != "Notebook"]
fila_media = {"Notebook": "MEDIA GLOBAL"}
for col in cols_r2:
    fila_media[col] = df_target[col].mean(skipna=True)
df_target = pd.concat([df_target, pd.DataFrame([fila_media])], ignore_index=True)

tablas_por_target[target] = df_target

html_target = render_tabla_html(df_target)

print("=" * 100)
print(f"5.2. TABLA COMPARATIVA POR TARGET: {target} (Base: {LABELS_RAMAS[base_branch]})")
print("=" * 100)
mostrar_tabla_html(df_target, f"{target} (Base: {LABELS_RAMAS[base_branch]})")

ruta_html_target = f"output/comparativas/por_target_{target}_{'_'}.join(BRANCHES)}.html"
with open(ruta_html_target, "w", encoding="utf-8") as f:
    f.write(html_target)
print(f"Guardado en: {ruta_html_target}\n")

html_partes.append(f"<h2>{target}</h2>\n{html_target}")

html_final = (
    "<html><head><meta charset='utf-8'>"
    + ESTILO_HTML_TABLAS
    + "</head><body>"
    + f"<h1>Comparativa por targets (Base: {LABELS_RAMAS[base_branch]})</h1>"
    + "\n".join(html_partes)
    + "</body></html>"
)

ruta_html_final = f"output/comparativas/todas_las_tablas_{'_'}.join(BRANCHES)}.html"
with open(ruta_html_final, "w", encoding="utf-8") as f:
    f.write(html_final)

print("=" * 100)
print(f"Export final con todas las tablas guardado en: {ruta_html_final}")
print("=" * 100)

```

```

=====
5.2. TABLA COMPARATIVA POR TARGET: nematode_shannon_z (Base: n_vars=34)
=====

```

nematode_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.0112	0.0063	0.0031	-0.0019	0.0073	0.0022	0.0045	-0.0006
rf_model.ipynb	0.1693	0.1684	0.1421	0.1335	0.1624	0.1366	0.1354	0.1278
rf_multisalida.ipynb	0.1545	0.1457	0.1429	0.1514	0.1493	0.1645	0.1541	0.1527
regressorchain.ipynb	0.1467	0.1435	0.1348	0.1352	0.1406	0.1433	0.1363	0.1314
xgboost_model.ipynb	0.1637	0.1679	0.1622	0.1461	0.1692	0.1600	0.1447	0.1492
xgb_multisalida.ipynb	0.2228	0.2193	0.2110	0.2163	0.2245	0.2339	0.2285	0.2189
mlp_multisalida.ipynb	0.0656	0.0585	0.1103	0.0457	0.1381	0.1238	0.0525	0.1144
mlp_custom_loss.ipynb	0.0520	0.0710	0.1200	0.0565	0.0667	0.0846	0.0327	0.0482
MEDIA GLOBAL	0.1232	0.1226	0.1283	0.1104	0.1323	0.1311	0.1111	0.1177

Guardado en: output/comparativas/por_target_nematode_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-1-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

```

=====
5.2. TABLA COMPARATIVA POR TARGET: macro_shannon_z (Base: n_vars=34)
=====

```

macro_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.1257	0.1174	0.1282	0.1191	0.1341	0.1254	0.1324	0.1231
rf_model.ipynb	0.3199	0.3082	0.3194	0.3141	0.3200	0.3158	0.3316	0.3231
rf_multisalida.ipynb	0.2000	0.1892	0.1894	0.1966	0.1936	0.2026	0.2036	0.2103
regressorchain.ipynb	0.2804	0.2708	0.2816	0.2757	0.2828	0.2818	0.2864	0.2931
xgboost_model.ipynb	0.2601	0.2643	0.2668	0.2582	0.2668	0.2578	0.2598	0.2697
xgb_multisalida.ipynb	0.3771	0.3650	0.3677	0.3795	0.3825	0.3958	0.3836	0.3894
mlp_multisalida.ipynb	0.2101	0.2066	0.2309	0.1935	0.2608	0.2491	0.1813	0.2525
mlp_custom_loss.ipynb	0.1817	0.2475	0.2364	0.2086	0.2300	0.2562	0.1934	0.1918
MEDIA GLOBAL	0.2444	0.2461	0.2526	0.2432	0.2588	0.2606	0.2465	0.2566

Guardado en: output/comparativas/por_target_macro_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-1-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

```

=====
5.2. TABLA COMPARATIVA POR TARGET: earthworm_shannon_z (Base: n_vars=34)
=====

```

earthworm_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.2395	0.2434	0.2401	0.2441	0.2515	0.2429	0.2515	0.2430
rf_model.ipynb	0.5041	0.5014	0.4995	0.5121	0.5012	0.5140	0.5108	0.5081
rf_multisalida.ipynb	0.4160	0.4045	0.4113	0.4249	0.4045	0.4235	0.4221	0.4178
regressorchain.ipynb	0.4876	0.4890	0.4812	0.4854	0.4906	0.4919	0.4849	0.4742
xgboost_model.ipynb	0.4946	0.4931	0.4906	0.4954	0.4935	0.4958	0.4965	0.5012
xgb_multisalida.ipynb	0.5375	0.5472	0.5423	0.5328	0.5377	0.5466	0.5296	0.5482
mlp_multisalida.ipynb	0.3583	0.3504	0.3463	0.3546	0.3602	0.4065	0.3565	0.4029
mlp_custom_loss.ipynb	0.3438	0.3569	0.4070	0.3835	0.3257	0.3519	0.3011	0.3776
MEDIA GLOBAL	0.4227	0.4232	0.4273	0.4291	0.4206	0.4341	0.4191	0.4341

Guardado en: output/comparativas/por_target_earthworm_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

=====

5.2. TABLA COMPARATIVA POR TARGET: orib_shannon_z (Base: n_vars=34)

=====

orib_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.1137	0.1012	0.1134	0.1007	0.1127	0.1007	0.1112	0.0989
rf_model.ipynb	0.1794	0.1807	0.1864	0.1890	0.1934	0.1905	0.1860	0.1985
rf_multisalida.ipynb	0.1693	0.1631	0.1645	0.1682	0.1642	0.1740	0.1716	0.1708
regressorchain.ipynb	0.2069	0.2103	0.2024	0.2024	0.2071	0.2044	0.2080	0.2119
xgboost_model.ipynb	0.1147	0.1242	0.1424	0.1500	0.1447	0.1518	0.1485	0.1446
xgb_multisalida.ipynb	0.2275	0.2194	0.2271	0.2245	0.2353	0.2397	0.2270	0.2400
mlp_multisalida.ipynb	0.2277	0.2358	0.2583	0.2269	0.2900	0.2701	0.2489	0.2846
mlp_custom_loss.ipynb	0.1990	0.2286	0.2716	0.2149	0.2601	0.2602	0.2712	0.2009
MEDIA GLOBAL	0.1798	0.1829	0.1958	0.1846	0.2009	0.1989	0.1966	0.1938

Guardado en: output/comparativas/por_target_orib_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

=====

5.2. TABLA COMPARATIVA POR TARGET: meso_shannon_z (Base: n_vars=34)

=====

meso_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	-0.2499	-0.2584	-0.2521	-0.2603	-0.2620	-0.2689	-0.2833	-0.2932
rf_model.ipynb	-0.1731	-0.1756	-0.1595	-0.2002	-0.1616	-0.2001	-0.2041	-0.1708
rf_multisalida.ipynb	-0.1178	-0.1183	-0.1178	-0.1272	-0.1166	-0.1245	-0.1279	-0.1251
regressorchain.ipynb	-0.1640	-0.1532	-0.1599	-0.1594	-0.1582	-0.1580	-0.1663	-0.1747
xgboost_model.ipynb	-0.1868	-0.2031	-0.2008	-0.2224	-0.2089	-0.2292	-0.2272	-0.2002
xgb_multisalida.ipynb	-0.2861	-0.2724	-0.2562	-0.2826	-0.2870	-0.3051	-0.2915	-0.3146
mlp_multisalida.ipynb	-0.2084	-0.2769	-0.4271	-0.2666	-0.3858	-0.3461	-0.4403	-0.2757
mlp_custom_loss.ipynb	-0.4116	-0.3330	-0.4716	-0.1979	-0.3474	-0.4391	-0.4098	-0.3789
MEDIA GLOBAL	-0.2247	-0.2239	-0.2556	-0.2146	-0.2409	-0.2589	-0.2688	-0.2417

Guardado en: output/comparativas/por_target_meso_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

=====

5.2. TABLA COMPARATIVA POR TARGET: coll_shannon_z (Base: n_vars=34)

=====

coll_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	-0.4998	-0.4927	-0.4816	-0.4768	-0.6035	-0.6117	-0.5358	-0.5990
rf_model.ipynb	-0.3489	-0.3833	-0.3319	-0.3545	-0.3363	-0.3482	-0.3486	-0.3649
rf_multisalida.ipynb	-0.1953	-0.1905	-0.1894	-0.1996	-0.1939	-0.1947	-0.2008	-0.1931
regressorchain.ipynb	-0.6087	-0.6621	-0.6707	-0.6594	-0.6818	-0.6652	-0.6694	-0.5494
xgboost_model.ipynb	-0.3312	-0.3148	-0.3445	-0.3097	-0.3073	-0.3141	-0.3173	-0.3515
xgb_multisalida.ipynb	-0.3229	-0.4397	-0.3899	-0.3913	-0.3800	-0.4294	-0.3942	-0.4247
mlp_multisalida.ipynb	-0.7628	-0.8362	-1.2277	-0.8208	-0.8674	-0.3868	-1.0076	-0.3535
mlp_custom_loss.ipynb	-1.2843	-0.8777	-1.0490	-0.8599	-0.9322	-1.1463	-1.3520	-0.9949
MEDIA GLOBAL	-0.5442	-0.5246	-0.5856	-0.5090	-0.5378	-0.5121	-0.6032	-0.4789

Guardado en: output/comparativas/por_target_coll_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-pr
ep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

5.2. TABLA COMPARATIVA POR TARGET: bac_shannon_z (Base: n_vars=34)

bac_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var- 1])	R2 (n_vars=33 [model-prep-var- 2])	R2 (n_vars=32 [model-prep-var-1- 2])	R2 (n_vars=33 [model-prep-var- 3])	R2 (n_vars=32 [model-prep-var-1- 3])	R2 (n_vars=32 [model-prep-var-2- 3])	R2 (n_vars=31)
reg_model.ipynb	-0.0812	-0.0587	-0.0807	-0.0585	-0.0590	-0.0647	-0.0680	-0.0677
rf_model.ipynb	0.0077	0.0214	0.0126	0.0179	0.0077	0.0173	0.0094	0.0219
rf_multisalida.ipynb	0.0290	0.0125	0.0103	0.0476	0.0102	0.0352	0.0355	0.0306
regressorchain.ipynb	0.0288	0.0286	0.0332	0.0288	0.0316	0.0278	0.0261	0.0129
xgboost_model.ipynb	0.0092	0.0174	0.0203	0.0212	0.0113	0.0209	0.0237	0.0200
xgb_multisalida.ipynb	-0.0151	-0.0371	-0.0073	-0.0142	-0.0419	-0.0348	-0.0180	-0.0361
mlp_multisalida.ipynb	0.0032	0.0101	-0.1108	-0.0023	-0.1435	-0.0463	-0.1180	-0.0543
mlp_custom_loss.ipynb	-0.0494	-0.0446	-0.1116	-0.0389	-0.0513	-0.1280	-0.0523	0.0113
MEDIA GLOBAL	-0.0085	-0.0063	-0.0292	0.0002	-0.0294	-0.0216	-0.0202	-0.0077

Guardado en: output/comparativas/por_target_bac_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-pre
p-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

5.2. TABLA COMPARATIVA POR TARGET: fun_shannon_z (Base: n_vars=34)

fun_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var- 1])	R2 (n_vars=33 [model-prep-var- 2])	R2 (n_vars=32 [model-prep-var-1- 2])	R2 (n_vars=33 [model-prep-var- 3])	R2 (n_vars=32 [model-prep-var-1- 3])	R2 (n_vars=32 [model-prep-var-2- 3])	R2 (n_vars=31)
reg_model.ipynb	-0.0161	-0.0123	-0.0090	-0.0065	-0.0150	-0.0113	-0.0113	-0.0083
rf_model.ipynb	-0.0455	-0.0345	-0.0297	-0.0413	-0.0228	-0.0312	-0.0309	-0.0410
rf_multisalida.ipynb	-0.0127	-0.0185	-0.0185	-0.0187	-0.0155	-0.0234	-0.0188	-0.0188
regressorchain.ipynb	-0.0219	-0.0343	-0.0239	-0.0344	-0.0205	-0.0300	-0.0236	-0.0370
xgboost_model.ipynb	-0.0206	-0.0214	-0.0368	-0.0483	-0.0139	-0.0409	-0.0373	-0.0237
xgb_multisalida.ipynb	-0.0502	-0.0566	-0.0590	-0.0580	-0.0192	-0.0434	-0.0374	-0.0439
mlp_multisalida.ipynb	-0.0615	-0.0511	-0.1139	-0.0501	-0.0663	-0.0580	-0.0976	-0.0559
mlp_custom_loss.ipynb	-0.0859	-0.0291	-0.1072	-0.0583	-0.0273	-0.0463	-0.0895	-0.0499
MEDIA GLOBAL	-0.0393	-0.0322	-0.0498	-0.0394	-0.0251	-0.0356	-0.0433	-0.0348

Guardado en: output/comparativas/por_target_fun_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-pre
p-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

5.2. TABLA COMPARATIVA POR TARGET: euk_shannon_z (Base: n_vars=34)

euk_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var- 1])	R2 (n_vars=33 [model-prep-var- 2])	R2 (n_vars=32 [model-prep-var-1- 2])	R2 (n_vars=33 [model-prep-var- 3])	R2 (n_vars=32 [model-prep-var-1- 3])	R2 (n_vars=32 [model-prep-var-2- 3])	R2 (n_vars=31)
reg_model.ipynb	0.0407	0.0532	0.0384	0.0531	0.0386	0.0530	0.0384	0.0525
rf_model.ipynb	0.1683	0.1504	0.1571	0.1511	0.1554	0.1526	0.1645	0.1668
rf_multisalida.ipynb	0.1378	0.1301	0.1354	0.1371	0.1345	0.1371	0.1348	0.1396
regressorchain.ipynb	0.1624	0.1479	0.1530	0.1442	0.1532	0.1468	0.1511	0.1605
xgboost_model.ipynb	0.0912	0.0979	0.1017	0.0862	0.1048	0.0879	0.0917	0.0882
xgb_multisalida.ipynb	0.0886	0.0960	0.1104	0.0715	0.1158	0.0853	0.0926	0.0940
mlp_multisalida.ipynb	0.1323	0.1341	0.1453	0.1359	0.1938	0.2099	0.1061	0.1955
mlp_custom_loss.ipynb	0.0952	0.1426	0.1089	0.1287	0.1191	0.1411	0.1256	0.1105
MEDIA GLOBAL	0.1146	0.1190	0.1188	0.1135	0.1269	0.1267	0.1131	0.1260

Guardado en: output/comparativas/por_target_euk_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-pre
p-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

5.2. TABLA COMPARATIVA POR TARGET: oomy_shannon_z (Base: n_vars=34)

oomy_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.0857	0.0947	0.0867	0.0954	0.0852	0.0942	0.0864	0.0949
rf_model.ipynb	0.0832	0.0884	0.0942	0.0719	0.0970	0.0862	0.0999	0.0905
rf_multisalida.ipynb	0.1252	0.1280	0.1280	0.1222	0.1248	0.1251	0.1233	0.1288
regressorchain.ipynb	0.0913	0.0947	0.0964	0.1009	0.0953	0.1013	0.1033	0.0904
xgboost_model.ipynb	0.0965	0.0949	0.0985	0.0988	0.0924	0.0955	0.1003	0.1011
xgb_multisalida.ipynb	0.0620	0.0614	0.0767	0.0700	0.0691	0.0630	0.0581	0.0620
mlp_multisalida.ipynb	0.2119	0.2016	0.1538	0.1940	0.0930	0.1022	0.1391	0.1017
mlp_custom_loss.ipynb	0.2024	0.0860	-0.0033	0.0822	0.0718	0.0868	0.1848	0.1902
MEDIA GLOBAL	0.1198	0.1062	0.0914	0.1044	0.0911	0.0943	0.1119	0.1074

Guardado en: output/comparativas/por_target_oomy_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

=====

5.2. TABLA COMPARATIVA POR TARGET: cerc_shannon_z (Base: n_vars=34)

=====

cerc_shannon_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.0155	0.0171	0.0195	0.0213	0.0230	0.0248	0.0273	0.0293
rf_model.ipynb	-0.0318	-0.0299	-0.0296	-0.0220	-0.0250	-0.0435	-0.0274	-0.0106
rf_multisalida.ipynb	-0.0041	-0.0021	0.0004	-0.0037	-0.0024	-0.0035	-0.0089	0.0039
regressorchain.ipynb	-0.0006	0.0012	0.0057	-0.0006	0.0151	0.0110	0.0092	0.0081
xgboost_model.ipynb	-0.0414	-0.0418	-0.0316	-0.0500	-0.0341	-0.0626	-0.0619	-0.0459
xgb_multisalida.ipynb	-0.0330	-0.0107	-0.0317	-0.0141	0.0184	0.0058	-0.0073	0.0214
mlp_multisalida.ipynb	-0.0668	-0.0501	-0.0812	-0.0454	-0.1046	-0.1040	-0.0859	-0.1203
mlp_custom_loss.ipynb	-0.1223	-0.0558	-0.1385	-0.1080	-0.0628	-0.1055	-0.0828	-0.0476
MEDIA GLOBAL	-0.0356	-0.0215	-0.0359	-0.0278	-0.0215	-0.0347	-0.0297	-0.0202

Guardado en: output/comparativas/por_target_cerc_shannon_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

=====

5.2. TABLA COMPARATIVA POR TARGET: macro_order_richness_z (Base: n_vars=34)

=====

macro_order_richness_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.1521	0.1446	0.1572	0.1488	0.1673	0.1587	0.1691	0.1598
rf_model.ipynb	0.3417	0.3221	0.3602	0.3604	0.3564	0.3558	0.3630	0.3370
rf_multisalida.ipynb	0.2532	0.2398	0.2404	0.2520	0.2449	0.2626	0.2609	0.2666
regressorchain.ipynb	0.4069	0.3976	0.4075	0.4091	0.4052	0.4090	0.4116	0.4128
xgboost_model.ipynb	0.3082	0.2987	0.3011	0.3095	0.3047	0.3116	0.3105	0.3128
xgb_multisalida.ipynb	0.4013	0.4193	0.4110	0.4259	0.4258	0.4342	0.4178	0.4450
mlp_multisalida.ipynb	0.2683	0.2506	0.3387	0.2366	0.3569	0.3126	0.2702	0.3373
mlp_custom_loss.ipynb	0.2420	0.3420	0.3443	0.2779	0.3105	0.3489	0.2675	0.2422
MEDIA GLOBAL	0.2967	0.3018	0.3201	0.3025	0.3215	0.3242	0.3088	0.3142

Guardado en: output/comparativas/por_target_macro_order_richness_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

=====

5.2. TABLA COMPARATIVA POR TARGET: earthworm_richness_z (Base: n_vars=34)

=====

earthworm_richness_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.2789	0.2750	0.2803	0.2761	0.2820	0.2724	0.2842	0.2775
rf_model.ipynb	0.5722	0.5698	0.5677	0.5594	0.5672	0.5608	0.5560	0.5620
rf_multisalida.ipynb	0.4509	0.4317	0.4375	0.4703	0.4326	0.4655	0.4636	0.4581
regressorchain.ipynb	0.5359	0.5297	0.5270	0.5283	0.5309	0.5313	0.5299	0.5242
xgboost_model.ipynb	0.5171	0.5138	0.5128	0.5155	0.5118	0.5174	0.5149	0.5124
xgb_multisalida.ipynb	0.5893	0.5975	0.5862	0.5905	0.5794	0.5829	0.5744	0.5960
mlp_multisalida.ipynb	0.4539	0.4316	0.4381	0.4457	0.4717	0.4998	0.4401	0.4809
mlp_custom_loss.ipynb	0.4543	0.4778	0.5043	0.4694	0.4467	0.4488	0.3913	0.4551
MEDIA GLOBAL	0.4816	0.4784	0.4817	0.4819	0.4778	0.4849	0.4693	0.4833

Guardado en: output/comparativas/por_target_earthworm_richness_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

5.2. TABLA COMPARATIVA POR TARGET: orib_species_richness_z (Base: n_vars=34)

orib_species_richness_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.0921	0.0973	0.1000	0.0971	0.0927	0.0902	0.0925	0.0901
rf_model.ipynb	0.2034	0.2252	0.1984	0.2108	0.1993	0.2119	0.2171	0.2017
rf_multisalida.ipynb	0.2386	0.2347	0.2390	0.2335	0.2399	0.2431	0.2373	0.2433
regressorchain.ipynb	0.2026	0.2109	0.2081	0.1976	0.2106	0.2019	0.1991	0.2023
xgboost_model.ipynb	0.1259	0.1622	0.1248	0.1608	0.1623	0.1637	0.1618	0.1588
xgb_multisalida.ipynb	0.2800	0.2903	0.2674	0.2827	0.2903	0.2810	0.2724	0.2850
mlp_multisalida.ipynb	0.2700	0.2729	0.3060	0.2646	0.3319	0.3311	0.2894	0.3360
mlp_custom_loss.ipynb	0.2477	0.2658	0.3227	0.2543	0.2839	0.2947	0.3065	0.2323
MEDIA GLOBAL	0.2075	0.2199	0.2208	0.2127	0.2264	0.2272	0.2220	0.2187

Guardado en: output/comparativas/por_target_orib_species_richness_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

5.2. TABLA COMPARATIVA POR TARGET: meso_species_richness_z (Base: n_vars=34)

meso_species_richness_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	-0.0959	-0.1066	-0.0959	-0.1064	-0.1044	-0.1137	-0.0980	-0.1077
rf_model.ipynb	-0.0438	-0.0340	-0.0254	-0.0560	-0.0255	-0.0573	-0.0598	-0.0319
rf_multisalida.ipynb	-0.0255	-0.0270	-0.0252	-0.0293	-0.0233	-0.0267	-0.0299	-0.0263
regressorchain.ipynb	-0.0400	-0.0358	-0.0274	-0.0465	-0.0236	-0.0457	-0.0458	-0.0457
xgboost_model.ipynb	-0.0592	-0.0756	-0.0763	-0.0818	-0.0774	-0.0812	-0.0838	-0.0776
xgb_multisalida.ipynb	-0.1146	-0.0960	-0.1131	-0.1050	-0.1255	-0.1185	-0.1259	-0.1187
mlp_multisalida.ipynb	-0.0625	-0.1081	-0.2250	-0.0976	-0.1870	-0.1688	-0.2159	-0.1064
mlp_custom_loss.ipynb	-0.2028	-0.1676	-0.2760	-0.0675	-0.1774	-0.2337	-0.2085	-0.1863
MEDIA GLOBAL	-0.0805	-0.0813	-0.1080	-0.0738	-0.0930	-0.1057	-0.1084	-0.0876

Guardado en: output/comparativas/por_target_meso_species_richness_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

5.2. TABLA COMPARATIVA POR TARGET: coll_species_richness_z (Base: n_vars=34)

coll_species_richness_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	-0.7341	-0.7464	-0.6943	-0.7082	-0.8110	-0.8295	-0.7845	-0.8035
rf_model.ipynb	-0.5484	-0.5718	-0.5554	-0.5527	-0.5349	-0.5611	-0.5570	-0.5690
rf_multisalida.ipynb	-0.2920	-0.2958	-0.2975	-0.3157	-0.2932	-0.3037	-0.3124	-0.3000
regressorchain.ipynb	-0.5799	-0.5625	-0.5585	-0.5671	-0.5619	-0.5646	-0.5719	-0.5392
xgboost_model.ipynb	-0.5803	-0.5691	-0.5632	-0.5685	-0.5692	-0.5708	-0.5801	-0.5636
xgb_multisalida.ipynb	-0.6465	-0.7284	-0.6839	-0.6938	-0.6588	-0.6581	-0.7130	-0.7314
mlp_multisalida.ipynb	-1.3183	-1.3722	-2.0257	-1.3332	-1.5265	-0.6997	-1.6277	-0.6478
mlp_custom_loss.ipynb	-2.1265	-1.3919	-1.8351	-1.4057	-1.5350	-1.9564	-2.3229	-1.5757
MEDIA GLOBAL	-0.8532	-0.7798	-0.9017	-0.7681	-0.8113	-0.7680	-0.9337	-0.7163

Guardado en: output/comparativas/por_target_coll_species_richness_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

5.2. TABLA COMPARATIVA POR TARGET: bac_asv_richness_z (Base: n_vars=34)

bac_asv_richness_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	-0.0761	-0.0838	-0.0747	-0.0826	-0.0826	-0.0853	-0.0833	-0.0861
rf_model.ipynb	0.0042	0.0070	0.0156	0.0146	0.0119	0.0180	0.0227	0.0154
rf_multisalida.ipynb	0.0044	-0.0080	-0.0070	0.0136	-0.0058	0.0131	0.0087	0.0050
regressorchain.ipynb	-0.0052	-0.0047	0.0002	-0.0073	0.0013	-0.0017	0.0019	-0.0113
xgboost_model.ipynb	0.0014	-0.0011	0.0049	0.0010	0.0082	0.0000	0.0176	-0.0060
xgb_multisalida.ipynb	0.0155	0.0106	0.0388	0.0260	0.0377	0.0286	0.0372	0.0217
mlp_multisalida.ipynb	-0.0443	-0.0438	-0.1418	-0.0402	-0.1499	-0.0904	-0.1432	-0.0929
mlp_custom_loss.ipynb	-0.0900	-0.0962	-0.1601	-0.1002	-0.0983	-0.1780	-0.0982	-0.0342
MEDIA GLOBAL	-0.0238	-0.0275	-0.0405	-0.0219	-0.0347	-0.0370	-0.0296	-0.0236

Guardado en: output/comparativas/por_target_bac_asv_richness_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

=====
5.2. TABLA COMPARATIVA POR TARGET: fun_asv_richness_z (Base: n_vars=34)
=====

fun_asv_richness_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.0087	0.0078	0.0091	0.0083	0.0090	0.0084	0.0092	0.0084
rf_model.ipynb	-0.0256	-0.0277	-0.0274	-0.0320	-0.0249	-0.0236	-0.0191	-0.0300
rf_multisalida.ipynb	-0.0068	-0.0047	-0.0064	-0.0058	-0.0067	-0.0069	-0.0052	-0.0055
regressorchain.ipynb	-0.0186	-0.0190	-0.0205	-0.0260	-0.0192	-0.0264	-0.0230	-0.0204
xgboost_model.ipynb	-0.0242	-0.0461	-0.0303	-0.0306	-0.0283	-0.0251	-0.0251	-0.0248
xgb_multisalida.ipynb	-0.0662	-0.0612	-0.0603	-0.0654	-0.0637	-0.0680	-0.0736	-0.0645
mlp_multisalida.ipynb	0.0101	0.0097	0.0171	0.0105	0.0399	0.0202	0.0116	0.0160
mlp_custom_loss.ipynb	0.0043	-0.0138	-0.0120	0.0078	-0.0041	0.0157	0.0108	0.0134
MEDIA GLOBAL	-0.0148	-0.0194	-0.0163	-0.0166	-0.0123	-0.0132	-0.0143	-0.0134

Guardado en: output/comparativas/por_target_fun_asv_richness_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

=====
5.2. TABLA COMPARATIVA POR TARGET: euk_asv_richness_z (Base: n_vars=34)
=====

euk_asv_richness_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.0400	0.0602	0.0354	0.0561	0.0333	0.0560	0.0357	0.0585
rf_model.ipynb	0.2520	0.2506	0.2545	0.2455	0.2538	0.2343	0.2636	0.2707
rf_multisalida.ipynb	0.1903	0.1763	0.1822	0.1937	0.1776	0.1885	0.1892	0.1835
regressorchain.ipynb	0.2690	0.2651	0.2731	0.2624	0.2773	0.2629	0.2757	0.2731
xgboost_model.ipynb	0.1671	0.1671	0.1717	0.1559	0.1711	0.1589	0.1644	0.1612
xgb_multisalida.ipynb	0.2233	0.2300	0.2339	0.2180	0.2266	0.2022	0.2254	0.2281
mlp_multisalida.ipynb	0.2201	0.2303	0.2363	0.2152	0.2746	0.3031	0.2254	0.3006
mlp_custom_loss.ipynb	0.2118	0.2414	0.2202	0.2378	0.2464	0.2009	0.2423	0.1999
MEDIA GLOBAL	0.1967	0.2026	0.2009	0.1981	0.2076	0.2009	0.2027	0.2094

Guardado en: output/comparativas/por_target_euk_asv_richness_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

=====
5.2. TABLA COMPARATIVA POR TARGET: oomy_asv_richness_z (Base: n_vars=34)
=====

oomy_asv_richness_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.1035	0.1124	0.1038	0.1124	0.0987	0.1086	0.0928	0.1017
rf_model.ipynb	0.1843	0.2011	0.1971	0.1849	0.1995	0.1990	0.2031	0.2014
rf_multisalida.ipynb	0.1917	0.1825	0.1845	0.1992	0.1829	0.2006	0.1986	0.1998
regressorchain.ipynb	0.2019	0.2015	0.2055	0.2041	0.2130	0.2097	0.2199	0.2058
xgboost_model.ipynb	0.1776	0.1854	0.1884	0.1859	0.1946	0.1910	0.1949	0.1917
xgb_multisalida.ipynb	0.1650	0.1794	0.1753	0.1728	0.1805	0.1788	0.1859	0.1875
mlp_multisalida.ipynb	0.0911	0.0935	0.0587	0.0812	0.0037	0.0422	0.0685	0.0339
mlp_custom_loss.ipynb	0.0849	0.0602	-0.0406	0.0727	0.0633	0.0169	0.0977	0.0905
MEDIA GLOBAL	0.1500	0.1520	0.1341	0.1517	0.1420	0.1434	0.1577	0.1515

Guardado en: output/comparativas/por_target_oomy_asv_richness_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

5.2. TABLA COMPARATIVA POR TARGET: cerc_asv_richness_z (Base: n_vars=34)

cerc_asv_richness_z (Base: n_vars=34)

Notebook	R2 (n_vars=34)	R2 (n_vars=33 [model-prep-var-1])	R2 (n_vars=33 [model-prep-var-2])	R2 (n_vars=32 [model-prep-var-1-2])	R2 (n_vars=33 [model-prep-var-3])	R2 (n_vars=32 [model-prep-var-1-3])	R2 (n_vars=32 [model-prep-var-2-3])	R2 (n_vars=31)
reg_model.ipynb	0.1019	0.0935	0.0985	0.0890	0.0941	0.0861	0.0953	0.0877
rf_model.ipynb	0.1250	0.1235	0.1212	0.1116	0.1202	0.1095	0.1056	0.1134
rf_multisalida.ipynb	0.1171	0.1149	0.1128	0.1217	0.1124	0.1157	0.1169	0.1210
regressorchain.ipynb	0.1171	0.1213	0.1213	0.1185	0.1172	0.1158	0.1106	0.1172
xgboost_model.ipynb	0.1227	0.1351	0.1248	0.1364	0.1172	0.1311	0.1143	0.1205
xgb_multisalida.ipynb	0.0983	0.1027	0.1022	0.1028	0.0949	0.0743	0.0831	0.0854
mlp_multisalida.ipynb	0.0380	0.0575	0.0430	0.0452	0.0157	0.0180	0.0305	0.0035
mlp_custom_loss.ipynb	-0.0008	0.0454	-0.0187	0.0214	0.0546	0.0307	-0.0141	0.0048
MEDIA GLOBAL	0.0899	0.0992	0.0881	0.0933	0.0908	0.0852	0.0803	0.0817

Guardado en: output/comparativas/por_target_cerc_asv_richness_z_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

Export final con todas las tablas guardado en: output/comparativas/todas_las_tablas_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.html

7. Ranking de modelos

Se calcula directamente a partir de las métricas ya extraídas. Rank 1 = mejor modelo (mayor R²) en esa rama/target.

```
In [29]: print("=" * 100)
print("7.1.- RANKING POR RAMA (R2 global, 1 = mejor modelo)")
print("=" * 100)

r2_global_cols = [c for c in df_general.columns if c.startswith("R2_Global (")]
ranking_global = df_general.set_index("Notebook")[r2_global_cols].rank(ascending=False, axis=0)
ranking_global.columns = [c.replace("R2_Global (", "").rstrip(")") for c in ranking_global.columns]

print(ranking_global.to_string())

resumen_ranking = pd.DataFrame({
    "Ranking medio": ranking_global.mean(axis=1).round(2),
    "Desviacion (estabilidad)": ranking_global.std(axis=1).round(2),
}).sort_values("Ranking medio")

print("\n" + "=" * 100)
print("7.2.- RANKING MEDIO Y ESTABILIDAD ENTRE RAMAS")
print("=" * 100)
print(resumen_ranking.to_string())

os.makedirs("output/comparativas", exist_ok=True)
ruta_ranking = f"output/comparativas/ranking_global_{'_vs_'}.join(BRANCHES)}.csv"
ranking_global.to_csv(ruta_ranking)
print(f"\nGuardado en: {ruta_ranking}")
```

7.1.- RANKING POR RAMA (R2 global, 1 = mejor modelo)

	n_vars=34	n_vars=33 [model-prep-var-1]	n_vars=33 [model-prep-var-2]	n_vars=32 [model-prep-var-1-2]	n_vars=33 [model-prep-var-1-3]	n_vars=32 [model-prep-var-2-3]	n_vars=31
Notebook							
reg_model.ipynb	7.0	7.0	6.0	7.0	6.0		7.0
rf_model.ipynb	2.0	2.0	2.0	1.0			2.0
rf_multisalida.ipynb	1.0	1.0	1.0	2.0			1.0
regressorchain.ipynb	4.0	3.0	3.5	4.0			4.0
xgboost_model.ipynb	5.0	5.0	5.0	5.0			5.0
xgb_multisalida.ipynb	3.0	4.0	3.5	3.0			3.0
mlp_multisalida.ipynb	6.0	6.0	7.0	6.0			6.0
mlp_custom_loss.ipynb	8.0	8.0	8.0	8.0			8.0

7.2.- RANKING MEDIO Y ESTABILIDAD ENTRE RAMAS

Notebook	Ranking medio	Desviacion (estabilidad)
rf_multisalida.ipynb	1.25	0.46
rf_model.ipynb	1.75	0.46
xgb_multisalida.ipynb	3.31	0.46
regressorchain.ipynb	3.69	0.46
xgboost_model.ipynb	5.00	0.00
mlp_multisalida.ipynb	6.38	0.52
reg_model.ipynb	6.62	0.52
mlp_custom_loss.ipynb	8.00	0.00

Guardado en: output/comparativas/ranking_global_model-prep-var_vs_model-prep-var-1_vs_model-prep-var-2_vs_model-prep-var-1-2_vs_model-prep-var-3_vs_model-prep-var-1-3_vs_model-prep-var-2-3_vs_model-prep-var-1-2-3.csv

8. Barrido de random_state

Dos ramas, cada una entrena un modelo por semilla dentro de un rango y guarda el resultado en un CSV propio (`MODELS_DIR/barrido_rs_<modelo>/<...>.csv`). Aquí se lee ese CSV directamente vía `git show`, no se parsea el notebook.

```
In [30]: barrido_por_rama = {}

for branch in BRANCHES_BARRIDO:
    print(f"Leyendo barridos de la rama: {branch}...")
    barrido_por_rama[branch] = {}
    conteos_rama = {}

    for nb_name in NOTEBOOKS:
        nb_json = obtener_notebook_desde_git(GIT_REPO_PATH, branch, nb_name, NOTEBOOKS_SUBDIR)
        if nb_json is None:
            continue
        ruta_csv = extraer_ruta_csv_barrido(nb_json)
        if ruta_csv is None:
            print(f" [AVISO] {nb_name}: no se encontró la celda de barrido (¿no tiene Prueba 3 todavía?)")
            continue

        df_csv = obtener_csv_desde_git(GIT_REPO_PATH, branch, ruta_csv)
        if df_csv is None:
            barrido_por_rama[branch][nb_name] = None
            continue

        n_real = len(df_csv)
        n_esperado = extraer_rango_esperado_rs(nb_json)
        conteos_rama[nb_name] = n_real

        etiqueta_check = ""
        if n_esperado is not None and n_real != n_esperado:
            etiqueta_check = f" [AVISO] se esperaban {n_esperado} semillas (RANDOM_STATES en el código), CSV tiene {n_real}"

        print(f" {nb_name}: {ruta_csv} → {n_real} semillas{etiqueta_check}")
        barrido_por_rama[branch][nb_name] = df_csv

    # dentro de la misma rama, todos los notebooks deberían barrer el mismo
    # rango de semillas -- si uno se queda corto respecto al resto, avisar
    if len(set(conteos_rama.values())) > 1:
        print(f" [AVISO] No todos los notebooks de '{branch}' tienen el mismo número de semillas: {conteos_rama}")
    print()
```

Leyendo barridos de la rama: model-prep-rs-group...

```
reg_model.ipynb: output/models/reg/barrido_rs_ridge/ridge_barrido_rs.csv → 101 semillas
rf_model.ipynb: output/models/rf/barrido_rs_rf/rf_barrido_rs.csv → 101 semillas
rf_multisalida.ipynb: output/models/rf_multi/barrido_rs_rf_multi/rf_multi_barrido_rs.csv → 101 semillas
regressorchain.ipynb: output/models/chain/barrido_rs_chain/chain_barrido_rs.csv → 101 semillas
xgboost_model.ipynb: output/models/xgb/barrido_rs_xgb/xgb_barrido_rs.csv → 101 semillas
xgb_multisalida.ipynb: output/models/xgb_multi/barrido_rs_xgb_multi/xgb_multi_barrido_rs.csv → 101 semillas
mlp_multisalida.ipynb: output/models/mlp/barrido_rs_mlp/mlp_barrido_rs.csv → 101 semillas
mlp_custom_loss.ipynb: output/models/mlp_custom/barrido_rs_mlp_custom/mlp_custom_barrido_rs.csv → 101 semillas
```

Leyendo barridos de la rama: model-prep-rs-group-1...

```
reg_model.ipynb: output/models/reg/barrido_rs_ridge/ridge_barrido_rs.csv → 491 semillas
rf_model.ipynb: output/models/rf/barrido_rs_rf/rf_barrido_rs.csv → 491 semillas
rf_multisalida.ipynb: output/models/rf_multi/barrido_rs_rf_multi/rf_multi_barrido_rs.csv → 491 semillas
regressorchain.ipynb: output/models/chain/barrido_rs_chain/chain_barrido_rs.csv → 491 semillas
xgboost_model.ipynb: output/models/xgb/barrido_rs_xgb/xgb_barrido_rs.csv → 491 semillas
xgb_multisalida.ipynb: output/models/xgb_multi/barrido_rs_xgb_multi/xgb_multi_barrido_rs.csv → 491 semillas
mlp_multisalida.ipynb: output/models/mlp/barrido_rs_mlp/mlp_barrido_rs.csv → 491 semillas
mlp_custom_loss.ipynb: output/models/mlp_custom/barrido_rs_mlp_custom/mlp_custom_barrido_rs.csv → 491 semillas
```

```
In [31]: def construir_df_largo_desde_csv():
    filas = []
    for branch, notebooks_dict in barrido_por_rama.items():
        for nb_name, df_csv in notebooks_dict.items():
            if df_csv is None or "random_state" not in df_csv.columns:
                continue
            r2_cols = [c for c in df_csv.columns if c.endswith("_r2")]
            for _, row in df_csv.iterrows():
                rs = row["random_state"]
                filas.append({
                    "Notebook": nb_name, "branch": branch, "random_state": rs,
                    "target": "GLOBAL", "r2": row[r2_cols].mean(),
                })
            for t in TARGETS_PRIORITARIOS:
                col = f"{t}_r2"
                if col in df_csv.columns:
                    filas.append({
                        "Notebook": nb_name, "branch": branch, "random_state": rs,
                        "target": t, "r2": row[col],
                    })
    return pd.DataFrame(filas)

df_barrido = construir_df_largo_desde_csv()
print(f"Filas recogidas: {len(df_barrido)}")
df_barrido.head()
```

Filas recogidas: 14208

```
Out[31]:
```

	Notebook	branch	random_state	target	r2
0	reg_model.ipynb	model-prep-rs-group	0.0	GLOBAL	-0.016377
1	reg_model.ipynb	model-prep-rs-group	0.0	earthworm_shannon_z	0.239544
2	reg_model.ipynb	model-prep-rs-group	0.0	earthworm_richness_z	0.278931
3	reg_model.ipynb	model-prep-rs-group	1.0	GLOBAL	-0.016377
4	reg_model.ipynb	model-prep-rs-group	1.0	earthworm_shannon_z	0.239544

```
In [32]: print("=" * 100)
print("8.1.- ESTADÍSTICA DEL BARRIDO (media ± desviación típica sobre las semillas, por rama)")
print("=" * 100)

os.makedirs("output/comparativas", exist_ok=True)

for target in ["GLOBAL"] + TARGETS_PRIORITARIOS:
    sub = df_barrido[df_barrido["target"] == target]
```

```

resumen = sub.groupby(["Notebook", "branch"])[ "r2"].agg(
    media="mean", desviacion="std", minimo="min", maximo="max", n="count"
).round(4).sort_values("media", ascending=False)
print(f"\n--- {target} ---")
print(resumen.to_string())
resumen.to_csv(f"output/comparativas/barrido_resumen_{target}.csv")

```

=====

8.1.- ESTADÍSTICA DEL BARRIDO (media ± desviación típica sobre las semillas, por rama)

=====

--- GLOBAL ---

Notebook	branch	media	desviacion	minimo	maximo	n
rf_multisalida.ipynb	model-prep-rs-group	0.0953	0.0030	0.0869	0.1032	101
	model-prep-rs-group-1	0.0953	0.0033	0.0839	0.1053	491
rf_model.ipynb	model-prep-rs-group-1	0.0892	0.0042	0.0768	0.1029	491
	model-prep-rs-group	0.0890	0.0040	0.0796	0.0996	101
xgb_multisalida.ipynb	model-prep-rs-group-1	0.0786	0.0049	0.0639	0.0949	491
	model-prep-rs-group	0.0780	0.0055	0.0635	0.0880	101
xgboost_model.ipynb	model-prep-rs-group-1	0.0655	0.0044	0.0541	0.0783	491
	model-prep-rs-group	0.0650	0.0043	0.0532	0.0752	101
regressorchain.ipynb	model-prep-rs-group-1	0.0643	0.0116	0.0341	0.0988	491
	model-prep-rs-group	0.0635	0.0104	0.0402	0.0976	101
reg_model.ipynb	model-prep-rs-group-1	-0.0164	0.0000	-0.0164	-0.0164	491
	model-prep-rs-group	-0.0164	0.0000	-0.0164	-0.0164	101
mlp_multisalida.ipynb	model-prep-rs-group	-0.0271	0.0342	-0.1196	0.0606	101
mlp_custom_loss.ipynb	model-prep-rs-group	-0.0495	0.0312	-0.1351	0.0185	101
	model-prep-rs-group-1	-0.0588	0.0348	-0.1737	0.0461	491
mlp_multisalida.ipynb	model-prep-rs-group-1	-0.0678	0.0516	-0.2609	0.0568	491

--- earthworm_shannon_z ---

Notebook	branch	media	desviacion	minimo	maximo	n
xgb_multisalida.ipynb	model-prep-rs-group	0.5343	0.0108	0.5032	0.5612	101
	model-prep-rs-group-1	0.5330	0.0111	0.4981	0.5688	491
rf_model.ipynb	model-prep-rs-group-1	0.5018	0.0104	0.4718	0.5302	491
	model-prep-rs-group	0.4990	0.0091	0.4720	0.5195	101
xgboost_model.ipynb	model-prep-rs-group	0.4921	0.0106	0.4594	0.5147	101
	model-prep-rs-group-1	0.4914	0.0111	0.4574	0.5208	491
regressorchain.ipynb	model-prep-rs-group-1	0.4792	0.0157	0.4307	0.5209	491
	model-prep-rs-group	0.4789	0.0144	0.4445	0.5134	101
rf_multisalida.ipynb	model-prep-rs-group-1	0.4142	0.0092	0.3876	0.4433	491
	model-prep-rs-group	0.4140	0.0091	0.3802	0.4350	101
mlp_multisalida.ipynb	model-prep-rs-group-1	0.3653	0.0484	0.1936	0.4770	491
mlp_custom_loss.ipynb	model-prep-rs-group-1	0.3464	0.0446	0.1304	0.4571	491
mlp_multisalida.ipynb	model-prep-rs-group	0.3452	0.0361	0.2589	0.4198	101
mlp_custom_loss.ipynb	model-prep-rs-group	0.3169	0.0466	0.1959	0.4450	101
reg_model.ipynb	model-prep-rs-group	0.2395	0.0000	0.2395	0.2395	101
	model-prep-rs-group-1	0.2395	0.0000	0.2395	0.2395	491

--- earthworm_richness_z ---

Notebook	branch	media	desviacion	minimo	maximo	n
xgb_multisalida.ipynb	model-prep-rs-group-1	0.5867	0.0127	0.5505	0.6251	491
	model-prep-rs-group	0.5866	0.0112	0.5597	0.6106	101
rf_model.ipynb	model-prep-rs-group	0.5643	0.0103	0.5426	0.5873	101
	model-prep-rs-group-1	0.5635	0.0106	0.5353	0.5900	491
regressorchain.ipynb	model-prep-rs-group-1	0.5279	0.0121	0.4924	0.5664	491
	model-prep-rs-group	0.5270	0.0118	0.5046	0.5590	101
xgboost_model.ipynb	model-prep-rs-group	0.5140	0.0114	0.4831	0.5456	101
	model-prep-rs-group-1	0.5135	0.0120	0.4775	0.5476	491
mlp_multisalida.ipynb	model-prep-rs-group-1	0.4592	0.0455	0.2597	0.5618	491
rf_multisalida.ipynb	model-prep-rs-group-1	0.4494	0.0086	0.4200	0.4808	491
	model-prep-rs-group	0.4493	0.0087	0.4178	0.4733	101
mlp_custom_loss.ipynb	model-prep-rs-group-1	0.4491	0.0402	0.2841	0.5469	491
mlp_multisalida.ipynb	model-prep-rs-group	0.4354	0.0325	0.3511	0.5132	101
mlp_custom_loss.ipynb	model-prep-rs-group	0.4185	0.0410	0.2991	0.5315	101
reg_model.ipynb	model-prep-rs-group	0.2789	0.0000	0.2789	0.2789	101
	model-prep-rs-group-1	0.2789	0.0000	0.2789	0.2789	491

```

In [33]: import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")

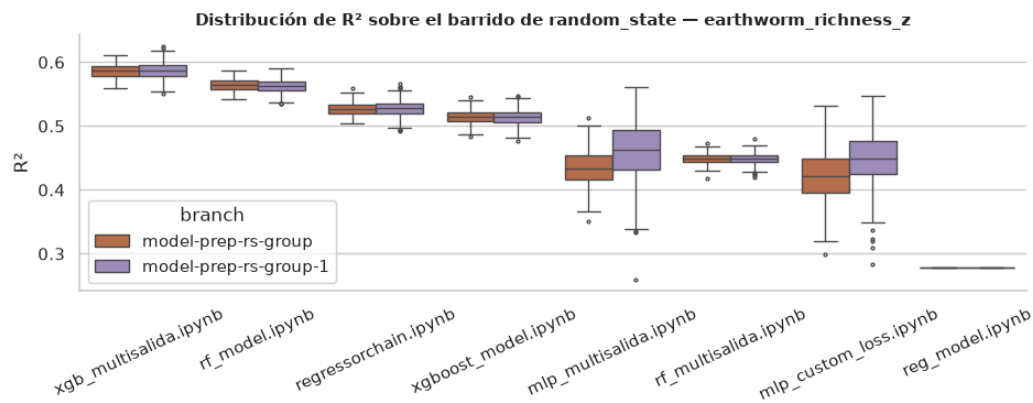
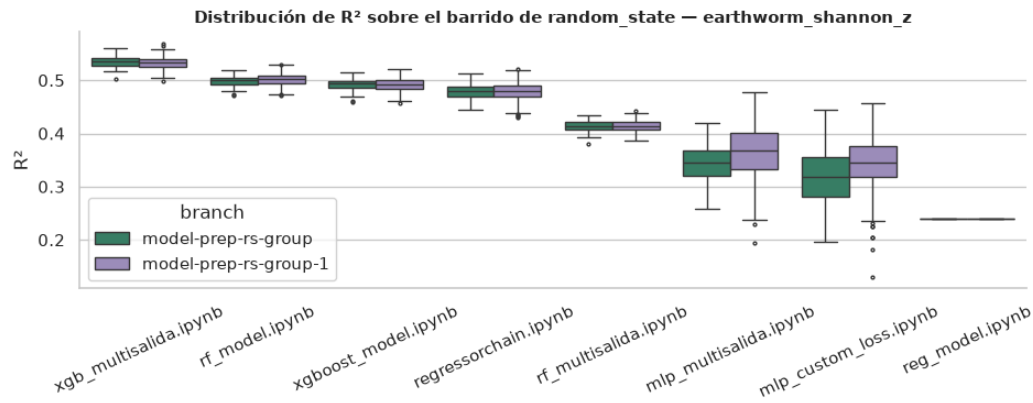
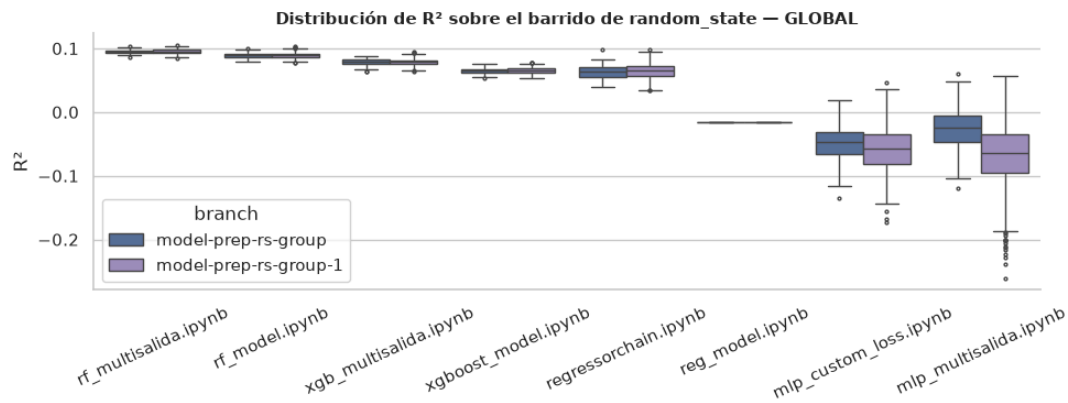
TARGET_COLORS = {"GLOBAL": "#4A6FA5", "earthworm_shannon_z": "#2E8B8B", "earthworm_richness_z": "#C9683A"}

targets_a_graficar = ["GLOBAL"] + TARGETS_PRIORITARIOS
fig, axes = plt.subplots(len(targets_a_graficar), 1, figsize=(10, 3.8 * len(targets_a_graficar)), squeeze=False)

for ax, target in zip(axes[:, 0], targets_a_graficar):
    sub = df_barrido[df_barrido["target"] == target]
    orden = sub.groupby("Notebook")["r2"].mean().sort_values(ascending=False).index
    sns.boxplot(data=sub, x="Notebook", y="r2", hue="branch", order=orden, ax=ax, fliersize=2,
        palette=[TARGET_COLORS.get(target, "#4A6FA5"), "#9885C4"])
    ax.set_title(f"Distribución de R² sobre el barrido de random_state - {target}", fontsize=11, fontweight="bold")
    ax.set_xlabel("")
    ax.set_ylabel("R²")
    ax.tick_params(axis="x", rotation=25)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

fig.tight_layout()
fig.savefig("output/comparativas/barrido_boxplot.png", dpi=160, bbox_inches="tight")
plt.show()

```



```
In [34]: print("=" * 100)
print("8.3.- RANKING MEDIO DE MODELOS A LO LARGO DE LAS SEMILLAS, POR RAMA")
print("=" * 100)

for branch in BRANCHES_BARRIDO:
    sub = df_barrido[(df_barrido["target"] == "GLOBAL") & (df_barrido["branch"] == branch)]
    pivot = sub.pivot(index="random_state", columns="Notebook", values="r2")
    ranking = pivot.rank(ascending=False, axis=1)
    resumen_rank = pd.DataFrame({
        "Ranking medio": ranking.mean().round(2),
        "Desviacion": ranking.std().round(2),
    }).sort_values("Ranking medio")
    print(f"\n--- {branch} ({len(pivot)} semillas) ---")
    print(resumen_rank.to_string())

=====
8.3.- RANKING MEDIO DE MODELOS A LO LARGO DE LAS SEMILLAS, POR RAMA
=====

--- model-prep-rs-group (101 semillas) ---
Ranking medio  Desviacion
Notebook
rf_multisalida.ipynb      1.11      0.34
rf_model.ipynb            1.96      0.37
xgb_multisalida.ipynb     3.09      0.45
xgboost_model.ipynb      4.38      0.56
regressorchain.ipynb     4.48      0.74
reg_model.ipynb          6.48      0.63
mlp_multisalida.ipynb    6.90      0.79
mlp_custom_loss.ipynb    7.61      0.60

--- model-prep-rs-group-1 (491 semillas) ---
Ranking medio  Desviacion
Notebook
rf_multisalida.ipynb      1.12      0.34
rf_model.ipynb            1.95      0.42
xgb_multisalida.ipynb     3.10      0.45
regressorchain.ipynb     4.38      0.76
xgboost_model.ipynb      4.44      0.55
reg_model.ipynb          6.25      0.51
mlp_custom_loss.ipynb    7.33      0.61
mlp_multisalida.ipynb    7.42      0.72
```